

The Loop Bug Field Guide

Examples, fixes and prevention tips for the six most common loop failures (Python)

Nearly every broken loop is one of these. You do not need to memorise them; use this as a checklist when something misbehaves.

Symptom	Likely bug	Where it hides
Program hangs, never finishes	Infinite loop	A while whose condition never becomes false
Last item missing / one short	Off-by-one	range end value, or a wrong comparison
One extra pass / crashes at the end	Off-by-one	<= where you needed <, or index past the end
Result is only the last value	Accumulator reset	Initialising inside the loop instead of before
Loop never runs	Empty or wrong-direction range	range(5, 5), or a condition false from the start
A value is skipped unexpectedly	Misplaced continue	continue jumping past an update

Tip: when a loop misbehaves, add **print()** inside it to show the loop variable and any accumulator on each pass. Most of these bugs become obvious within a few lines of output.

1. Program hangs, never finishes

Likely bug: Infinite loop | **Where it hides:** A while whose condition never becomes false

Buggy code

```
count = 0
while count < 5:
    print(count)
# count never changes, so count < 5 is always True
# Output: 0 0 0 0 0 0 ... (forever)
```

Fixed code

```
count = 0
while count < 5:
    print(count)
    count += 1      # moves the loop toward its end
# Output: 0 1 2 3 4
```

Why it happens and how to avoid it

A **while** loop only stops when its condition turns False. If nothing inside the body changes the variable the condition checks, it can never stop. **Avoid it:** before running, ask "what line in the body makes the condition False eventually?" If you can't point to one, add it. Prefer a **for** loop when you know how many passes you need.

2. Last item missing / one short

Likely bug: Off-by-one | **Where it hides:** range end value, or a wrong comparison

Buggy code

```
# Goal: print 1 to 5
for i in range(1, 5):
    print(i)
# Output: 1 2 3 4      <- the 5 is missing
```

Fixed code

```
for i in range(1, 6):      # or range(1, n + 1)
    print(i)
# Output: 1 2 3 4 5
```

Why it happens and how to avoid it

The end value of **range()** is *exclusive*: it stops just before it. **Avoid it:** to include n, write **range(start, n + 1)**. Quick check: **range(a, b)** gives exactly **b - a** numbers, so count them to confirm.

3. One extra pass / crashes at the end

Likely bug: Off-by-one | **Where it hides:** `<=` where you needed `<`, or index past the end

Buggy code

```
names = ["Asha", "Ben", "Chen"]    # valid indexes: 0, 1, 2
i = 0
while i <= len(names):              # len is 3, so i reaches 3
    print(names[i])
    i += 1
# Output: Asha Ben Chen
# IndexError: list index out of range
```

Fixed code

```
names = ["Asha", "Ben", "Chen"]
i = 0
while i < len(names):               # strictly less than
    print(names[i])
    i += 1

# Even safer: no index needed at all
for name in names:
    print(name)
```

Why it happens and how to avoid it

Indexes start at 0, so the last valid index is **len(list) - 1**. Using `<=` with **len()** gives one pass too many.

Avoid it: use `<` with **len()**, or loop directly over the items (**for name in names**) so there is no index to get wrong.

4. Result is only the last value

Likely bug: Accumulator reset | **Where it hides:** Initialising inside the loop instead of before

Buggy code

```
total = 0
for n in [10, 20, 30]:
    total = 0                # reset on EVERY pass
    total += n
print(total)                 # prints 30, expected 60
```

Fixed code

```
total = 0                    # set up ONCE, before the loop
for n in [10, 20, 30]:
    total += n
print(total)                 # prints 60
```

Why it happens and how to avoid it

An accumulator (a running total, count or list) must survive between passes. Code inside the loop runs every time, so setting it there wipes the previous work. **Avoid it:** initialise before the loop, update inside it, and read the result after it. This applies to **count = 0** and **items = []** too.

5. Loop never runs

Likely bug: Empty or wrong-direction range | **Where it hides:** range(5, 5), or a condition false from the start

Buggy code

```
for i in range(5, 1):      # counting up from 5 to 1? impossible
    print(i)
# Output: (nothing)

level = 5
while level > 10:          # already False on the first check
    print("Running")
# Output: (nothing)
```

Fixed code

```
for i in range(5, 1, -1): # step of -1 counts downwards
    print(i)
# Output: 5 4 3 2

level = 5
while level < 10:         # condition is True at the start
    print("Running")
    level += 1
```

Why it happens and how to avoid it

A **range** where start is already at or past the end (with a positive step) is empty, and a **while** condition that is False on the first check skips the body completely. No error is shown, which makes it easy to miss.

Avoid it: to count down, give a negative step. For a **while**, test the condition with your starting values in your head (or with a print) before the first pass.

6. A value is skipped unexpectedly

Likely bug: Misplaced continue | **Where it hides:** continue jumping past an update

Buggy code

```
total = 0
for n in range(1, 6):
    if n % 2 == 0:
        continue          # jumps to next n, skipping everything below
    print(n)
    total += n             # evens never reach this line
print(total)              # prints 9, but 1+2+3+4+5 = 15

# Worse in a while loop:
i = 0
while i < 5:
    if i == 2:
        continue          # i += 1 is skipped -> stuck at 2 forever
    i += 1
```

Fixed code

```
total = 0
for n in range(1, 6):
    total += n             # update FIRST, before any continue
    if n % 2 == 0:
        continue
    print(n)              # only odd numbers are printed
print(total)              # prints 15

i = 0
while i < 5:
    i += 1                 # always update before continue
    if i == 2:
        continue
```

Why it happens and how to avoid it

continue immediately jumps to the next pass, so every line below it is skipped for that pass. If an important update (a total, or a counter like **i += 1**) sits below it, it gets skipped too. **Avoid it:** put essential updates *above* the **continue**, or restructure with an **if / else** so nothing needs skipping.

Quick debugging checklist

- Does anything in a **while** body change the variable in its condition?
- Does **range(a, b)** include the last value you need? (b is excluded)
- Is every index between **0** and **len(list) - 1**?
- Are totals, counters and lists created *before* the loop?
- Could the loop condition or range be empty from the very start?
- Is any **continue** skipping a line that must always run?